

Contradiction-Aware Editing in Multi-Turn Settings for Graph Language Models

Arihant Rastogi
2023111012

Prasoon Dev
2023111014

Yajat Lakhanpal
2023111015

Hrishiraj Mitra
2023111037

Abstract

Graph Language Models (GLMs) condition answers on an input graph G , which existing work treats as fixed. That breaks once the setting is conversational: if a user revises a fact in language, the encoder still reads the old graph. We do not propose a new architecture; we wrap an existing GLM in a loop that emits a small edit, applies it to G , and re-encodes before the next question. Multi-turn revision then produces a failure that loop cannot handle: a later utterance incompatible with an edit already committed. We define such a contradiction as unsatisfiability of the active constraint set, localize the earlier edit responsible, and ask which revision to keep rather than preferring the newest. We will evaluate on synthetic sessions where the gold resolution is deliberately *not* the most recent revision.

1 Introduction

A GLM answers questions from an input graph G , usually as a single pair (G, q) . In a conversation a user may correct a fact: the language model sees the correction, but the graph encoder still reads the original G , so later answers can be grounded in a stale graph.

Plenz and Frank (2024) introduced GLMs so structure and text are encoded together. He et al. (2024) describe a conversational interface, but GraphQA is still single-shot. TEA-GLM (Wang et al., 2024), GraphToken (Perozzi et al., 2024), and CLEGR (Petkar et al., 2026) likewise treat G as fixed, and agentic graph token reasoning (Peng and Yang, 2026) only re-chooses a view of it. GraphRAG keeps state in an external store and SKG-Eval (Shil and Samui, 2026) scores dialogue against a parallel KG, but graph tokens sit inside the forward pass: neither mutates the tensors the GNN encodes.

Edits are also cumulative: turn three applies to G_2 , never to the original graph. That introduces a

second failure, because revision dialogue routinely produces a later utterance that *cannot* sit on top of an earlier one. If turn 2 closes station B and turn 5 puts through-service on the B–C line, turn 5 is neither a no-op nor a clean reversal, yet a write path with no notion of conflict silently takes the newer edit and drops the closure. We propose a write path that knows when a revision contradicts an earlier one, and asks.

2 What Counts as a Contradiction

“Contradiction” is overloaded. We take it from belief revision and ontology debugging, where a sentence set is inconsistent when it has no model, and ground it on the graph. Let each revision r denote constraints $I(r)$ over attributed graphs — the implied state of the edit, not the wording of the utterance — let Inv be the stated invariants (schema plus task invariants such as connectivity or mutual exclusion), and let R_{act} be the revisions not yet superseded, so that $T = \bigcup_{r \in R_{\text{act}}} I(r) \cup \text{Inv}$. A *conflict* holds iff T has no model, and the *culprit set* is a minimal unsatisfiable subset of T . A conflict is a **contradiction** while no culprit has been retired, and a **revision** if the user intends the new r to supersede some r_j in the culprit set.

This makes the DECODE judgment (Nie et al., 2021) decidable, and it is inter-context conflict (Xu et al., 2024) with committed edits in place of retrieved passages; localization corresponds to DECODE’s evidence indices, as in ontology debugging (Shchekotykhin et al., 2012). Crucially, unsatisfiability is the conflict, but whether it is a contradiction to surface or a revision to absorb depends on which constraints remain *active*, which is derivable neither from G nor from recency. That is what we propose to ask. Three cases arise: **invariant violations**, caught by the schema check; **silent incompatibilities**, solver-decidable given Inv ; and **intent conflicts**, valid graphs that still cannot both

be what the user meant.

3 Method

A GLM forward pass cannot rewrite G inside the GNN, so we wrap an existing model (G-Retriever, GraphToken, or TEA-GLM) in an outer loop. On each turn we prompt the GLM with the current graph and u_t to emit an edit Δ , parse it (illegal edits become no-ops), apply it with a schema-checked function, then re-encode G and answer q_t .

The model has no tools. It already emits text, and we restrict that text to four operations: SET a node attribute, ADD an edge, DEL an edge, or NOOP. Two calls, same weights, and the QA prompt is only (G, q_t) — the revision utterance is withheld, so if QA succeeds the fact is genuinely in the graph. Constrained decoding or light LoRA is a fallback only if writing the edit is the bottleneck. Skipping re-encoding makes this GraphRAG: it is a GLM method only if the apply step precedes the next GNN forward pass.

Around this loop we add four stages. **Detect**: a conflict gate before commit, using the schema check for invariant violations and a tentative apply plus invariant test for silent ones. **Attribute**: on a conflict at turn k , replay $G_0 \dots G_{k-1}$ and isolate the minimal earlier edit set that makes u_k unsatisfiable — a decision procedure over a typed log, not a learned component. **Ask**: one discriminating question naming that edit. **Repair**: retire the loser, replay the later edits that still apply, and re-encode.

Detection and localization are machinery, not the contribution: every strategy we know of resolves by recency, instruction-tuned models prefer the latest instruction (Zhang et al., 2025), and models detect conflicting instructions yet rarely disclose them (He et al., 2025). Asking earns its place on items where **recency is wrong** and the earlier edit should be preserved.

4 Dataset

Items come from a synthetic graph in the style of CLEGR: a fictional metro with invented station names and attributes, so parametric memory cannot skip G . The generator samples a graph G_0 of 15 to 30 nodes with attributes such as open or closed, and two paths between a query pair so that closing a node changes the answer. It fixes an executable query — say the shortest path through open stations — samples SET, ADD, and DEL operations that change that answer, and verbalizes each as an

utterance. Gold answers come from a solver. We discard an item if the question is answerable without the graph, if the utterance plus question recover the answer, or if G_0 already yields the new answer.

For contradictions the generator must stop drawing every edit from the current graph. To plant a conflict at turn k against turn $j < k$, we sample the later edit from an ancestor state G_{j-1} , so it is coherent alone but incompatible with what the log has committed. Each item carries the conflicting turn index, the gold culprit set, the intended graph, the stated invariant where needed, and a `keep_decision` in {earlier, later, amend}. Gold answers stay solver-computable *after* the decision: replay the repaired log and solve on the intended graph. Only `keep_decision` is planted rather than derived, the one label neither the graph nor the log supplies; simulated user replies come from it, so no human is needed at evaluation. The **recency-wrong subset** is the items where `keep_decision` is not *later*; without it the rest is not worth generating.

5 Evaluation

A frozen graph should keep the old answer after a revision; if it does not, the item is too easy. Applying the gold edit and re-encoding, then asking q_t with no revision text, checks the GLM can read an updated graph; if prompting the revision in text over a frozen G_0 matches that oracle, rewriting G is unnecessary. We compare the self-edit loop against a GraphRAG-style baseline that never re-encodes.

We add four contradiction baselines, reported even where they should win. A **symbolic solver** over the edit log replays, checks invariants, and enumerates culprit sets, and should be near-exact on the first two cases; we include it to make clear we claim neither detection nor localization as a result. A **transcript-only LLM** reads a frozen graph plus the history; **last-write-wins** commits the newest edit and fails by construction on the recency-wrong subset; **oracle attribution** supplies the true culprit set, so if accuracy does not move, localization is not the bottleneck. Beyond exact match and edit correctness we report detection, localization, questions asked, and post-repair state exactness against the intended graph, and that exactness on the recency-wrong subset — the decisive number. One risk: if contradictions are imperceptible through graph tokens, detection falls to the symbolic gate; we will probe this with an oracle-injected violation.

References

- Xiaoxin He, Yijun Tian, Yifei Sun, Nitesh V. Chawla, Thomas Laurent, Yann LeCun, Xavier Bresson, and Bryan Hooi. 2024. G-Retriever: Retrieval-augmented generation for textual graph understanding and question answering. In *Advances in Neural Information Processing Systems*.
- Xingwei He, Qianru Zhang, A-Long Jin, Yuan Ma, Zhenghao Yuan, and Siu Ming Yiu. 2025. [ConInstruct: Evaluating large language models on conflict detection and resolution in instructions](#). *Preprint*, arXiv:2511.14342.
- Yixin Nie, Mary Williamson, Mohit Bansal, Douwe Kiela, and Jason Weston. 2021. I like fish, especially dolphins: Addressing contradictions in dialogue modeling. In *Proceedings of the 59th Annual Meeting of the Association for Computational Linguistics and the 11th International Joint Conference on Natural Language Processing (Volume 1: Long Papers)*, pages 1699–1713. Association for Computational Linguistics.
- Zhuoyi Peng and Yi Yang. 2026. [Agentic graph token reasoning](#). *Preprint*, arXiv:2608.00542.
- Bryan Perozzi, Bahare Fatemi, Dustin Zelle, Anton Tsitsulin, Mehran Kazemi, Rami Al-Rfou, and Jonathan Halcrow. 2024. [Let your graph do the talking: Encoding structured data for LLMs](#). *Preprint*, arXiv:2402.05862.
- Soham Petkar, Hari Aakash K, Anirudh Vempati, Akshit Sinha, Ponnurangam Kumaraguru, and Chirag Agarwal. 2026. A graph talks, but who’s listening? rethinking evaluations for graph-language models. In *Findings of the Association for Computational Linguistics: ACL 2026*, pages 32441–32462, San Diego, California, United States. Association for Computational Linguistics.
- Moritz Plenz and Anette Frank. 2024. Graph language models. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 4477–4494, Bangkok, Thailand. Association for Computational Linguistics.
- Kostyantyn Shchekotykhin, Gerhard Friedrich, Philipp Fleiss, and Patrick Rodler. 2012. Interactive ontology debugging: Two query strategies for efficient fault localization. *Journal of Web Semantics*, 12–13:88–103.
- Avijit Shil and Suman Samui. 2026. [SKG-Eval: Stateful evaluation of multi-turn dialogue via incremental semantic knowledge graphs](#). *Preprint*, arXiv:2605.16650.
- Duo Wang, Yuan Zuo, Fengzhi Li, and Junjie Wu. 2024. LLMs as zero-shot graph learners: Alignment of GNN representations with LLM token embeddings. In *Advances in Neural Information Processing Systems*.
- Rongwu Xu, Zehan Qi, Zhijiang Guo, Cunxiang Wang, Hongru Wang, Yue Zhang, and Wei Xu. 2024. Knowledge conflicts for LLMs: A survey. In *Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing*, pages 8541–8565, Miami, Florida, USA. Association for Computational Linguistics.
- Zhihan Zhang, Shiyang Li, Zixuan Zhang, Xin Liu, Haoming Jiang, Xianfeng Tang, Yifan Gao, Zheng Li, Haodong Wang, Zhaoxuan Tan, Yichuan Li, Qingyu Yin, Bing Yin, and Meng Jiang. 2025. IHEval: Evaluating language models on following the instruction hierarchy. In *Proceedings of the 2025 Conference of the Nations of the Americas Chapter of the Association for Computational Linguistics: Human Language Technologies (Volume 1: Long Papers)*, pages 8374–8398. Association for Computational Linguistics.